

Madhav Sai Nalabothu

Embedded Firmware Engineer: Bare-metal ARM, FreeRTOS, Connected Devices

Stony Brook, NY (Open to relocation) | [linkedin.com/in/nmadhav-sai](https://www.linkedin.com/in/nmadhav-sai) | madhavsai.nalabothu@gmail.com | (586) 205-1797 | [GitHub](#)

EDUCATION

Stony Brook University, New York

Jan 2024 - Jan 2026

Master of Science in Computer Engineering

GPA: 3.89/4.0

Coursework: Embedded Microcontroller Systems II, Real-time Embedded Systems, Secure IoT Embedded Systems, Operating Systems, Pattern Recognition, Deep Learning, Computer Vision

Amrita Vishwa Vidyapeetham, India

Jul 2019 - Aug 2023

B.Tech in Electrical and Electronics Engineering (First Class with Distinction)

WORK EXPERIENCE

Graduate Research Assistant | Stony Brook University

Aug 2024 - Jan 2025

- Deployed INT8-quantized TFLite gesture recognition model on Raspberry Pi 4 (Arducam IMX708 12MP via libcamera), achieving 0.06ms inference latency at 7.2 FPS fully offline, 6x faster than sklearn baseline.
- Diagnosed a training-deployment gap between Mac webcam (training data) and RPi camera (deployment data) causing live accuracy to drop from 98% (test) to ~60% (runtime); retrained with device-specific data, restoring runtime accuracy to 95%+.
- Built 5-class gesture pipeline extracting 21 hand keypoints per frame with wrist-relative normalization and mirror augmentation; trained a 2-layer MLP classifier reaching 98% test accuracy.

Embedded Software Engineer | Coramandel Infrastructure Pvt. Ltd.

Nov 2022 - Dec 2023

- Developed motor protection firmware on STM32 (ARM Cortex-M, STM32 HAL, C) for a pump station, sampling motor current via timer-based ADC ISRs every 50ms and temperature every 500ms. Threshold-based fault logic triggered GPIO relay shutoff within 200ms, preventing induction motor burnout from overload and dry-run conditions.
- Built unit tests in C++ (GoogleTest) with FFF to mock STM32 HAL GPIO and ADC calls, enabling firmware validation without hardware-in-the-loop (HIL), detecting 2 logic bugs across overload, dry-run, and debounce scenarios pre-deployment.
- Configured peripheral drivers (ADC, GPIO, timers) and ISR handlers. Debugged with JTAG and logic analyzer during board bring-up.

TECHNICAL SKILLS

Languages: C, C++, Python, ARM Assembly, MATLAB

RTOS & Architectures: FreeRTOS, Zephyr, ThreadX, Linux, bare-metal, ARM Cortex-M (ARMv7-M), ESP32, STM32, RPi 4

Peripherals & Protocols: I2C, SPI, UART, CAN, DMA, ADC, DAC, PWM, GPIO, Timers, BLE, Wi-Fi

Networking & Cloud: MQTT, HTTPS, TCP/IP, mTLS, X.509, AWS IoT Core, OTA updates

Tools & Build: ESP-IDF, STM32CubeIDE, GCC, GDB, QEMU, CMake, Make, Git, linker scripts, CI/CD

Testing & Debug: GoogleTest, FFF, JTAG/SWD, oscilloscope, logic analyzer, serial tracing

PROJECTS

[Context Swap](#) | C, ARM Assembly, bare-metal, ARMv7-M, linker scripts, Makefile

- Prevented priority inversion by routing all context switches through PendSV at lowest priority 0xFF, keeping SysTick solely responsible for pending the switch so no ISR is preempted mid-execution.
- Isolated 3 concurrent task stacks per ARM-M3 dual frame convention: software saving r4 - r11 via stmdb/ldmia in assembly and hardware saved frame left to CPU exception return mechanism.

[The ESP32 Sentinel](#) | FreeRTOS, ESP-IDF, AWS IoT Core, MQTT, mTLS, OTA

- Eliminated missed PIR (HC-SR501) sensor events during rapid successive interrupts by separating interrupt handling from MQTT publishing, with a 16-deep ISR-safe FreeRTOS queue.
- Prevented unauthorized access by authenticating each device to AWS IoT Core through unique X.509 certificates embedded into flash, establishing mutual TLS in <2s.
- Implemented remote OTA firmware updates over MQTT, downloading firmware from S3 over HTTPS with automatic bootloader rollback on failure, recovering full operation in <35s.

[Vital Stack](#) | ESP32, FreeRTOS, I2C, SPI, UART, DMA

- Eliminated polling by wiring MAX30102 INT pin to a GPIO ISR releasing a binary semaphore, unblocking a dedicated FreeRTOS task per sensor for MPU6050 and MAX30102 to capture BPM, SpO2, and 3-axis acceleration.
- Prevented sample loss during peak acquisition by routing SPI writes to W25Q64 flash via DMA, buffering into a ring buffer and bulk offloading to FAT32 SD card between sessions.
- Streamed timestamped frames over UART via CP2102 to a Python listener, framing each packet with header, sensor ID, payload, and CRC checksum to detect corrupted transmissions.